

Elliptic Curve Cryptography (ECC)

Aim

To implement the **Elliptic Curve Cryptography (ECC)** algorithm for secure key generation, encryption, and decryption using elliptic curve arithmetic, providing high security with smaller key sizes.

Algorithm

Key Generation

Step 1: Start the program.

Step 2: Select the elliptic curve parameters:

- Prime number (**p**)
- Curve equation

$$y^2 = x^3 + ax + b \pmod{p}$$

- Base point (**G**)

Step 3: Choose a private key (**d**) such that $1 < d < n$ where **n** is the order of the base point.

Step 4: Compute the public key. $Q = d \times G$

Step 5: Publish the public key **Q** and keep the private key **d** secret.

Encryption

Step 6: Represent the plaintext message as a point **M** on the elliptic curve.

Step 7: Choose a random integer **k**.

Step 8: Compute $C_1 = kG$

Step 9: Compute $C_2 = M + kQ$

Step 10: The ciphertext is (C_1, C_2)

Decryption

Step 11: Compute $M = C_2 - dC_1$

Step 12: Recover the original plaintext.

Step 13: Stop the program.

C Program (Simplified ECC Demonstration)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int p = 17;
```

```
    int G = 5;
```

```

int d;

int Q;

int k;

int M;

int C1, C2;

int decrypted;

printf("Enter Private Key: ");

scanf("%d", &d);

printf("Enter Message (Integer): ");

scanf("%d", &M);

printf("Enter Random Key (k): ");

scanf("%d", &k);

Q = (d * G) % p;

C1 = (k * G) % p;

C2 = (M + (k * Q)) % p;

decrypted = (C2 - (d * C1)) % p;

if(decrypted < 0)

    decrypted += p

printf("\nPublic Key = %d", Q);

printf("\nCipher Text = (%d,%d)", C1, C2);

printf("\nDecrypted Message = %d\n", decrypted);

return 0;}

```

OUTPUT :

The screenshot shows an IDE window titled "D:\CSA5112 - Cryptography\LAB PROGRAMS\ECC.cpp - [Executing] - Embarcadero Dev-C++ 6.3". The code in the editor is as follows:

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int p = 17;
6     int G = 5;
7     int d;
8     int Q;
9     int k;
10    int M;
11    int C1, C2;
12    int decrypted;
13
14    printf("Enter Private Key: ");
15    scanf("%d", &d);
16
17    printf("Enter Message (Integer): ");
18    scanf("%d", &M);
19
20    printf("Enter Random Key (k): ");
21    scanf("%d", &k);
22
23    // Public Key
24    Q = (d * G) % p;
25
26    // Encryption
27    C1 = (k * G) % p;
28    C2 = (M + (k * Q)) % p;
29
30    // Decryption
31    decrypted = (C2 - (d * C1)) % p;
32
33    if(decrypted < 0)
34        decrypted += p;
35
36    printf("\nPublic Key = %d", Q);

```

The console output shows the following results:

```

Enter Private Key: 7
Enter Message (Integer): 10
Enter Random Key (k): 3

Public Key = 1
Cipher Text = (15,13)
Decrypted Message = 10

-----
Process exited after 13.55 seconds with return value 0
Press any key to continue . . .

```

The IDE also shows the compiler output at the bottom, indicating that the program compiled successfully with no errors or warnings.

